

IMPERIAL COLLEGE LONDON
DEPARTMENT OF AERONAUTICS

Group Design Project Final Report: Autonomous Water Sampling

Project Director: Dr. Mirko Kovac
Date: June 19, 2017
Name: Dario Arcos J.
CID: 00949920
Sub-Team BAC04 - Sensors Pack
Supervisor Thulasi Mylvaganam
Second Marker Aditya Paranjape

Imperial College London
South Kensington Campus
SW7 2AZ
United Kingdom

Abstract

In this paper, we present and develop the design and manufacturing of a fully integrated electrical and computing system for the sensor pack carried by the UAV (unmanned aerial vehicle), which is responsible for the monitoring and sampling of the water. The possible scenarios in which the use of a drone for this purpose would be beneficial were studied and the sensors in charge of sensing the water were selected accordingly. Furthermore, possible options for controlling the depth at which the monitoring would take place were carefully analyzed and a clamping mechanism actuating according to the readings of a depth sensor was found to be the optimal solution.

After studying different methods for sampling the water, it was decided to use a peristaltic pump controlled by the Arduino Mega board, the core of the electrical system. Having selected the main components of the system, we then proceeded to elaborate the main algorithms involved and build the software in accordance. At the same time, different options for the live data transmission were analyzed, showing that sending the information first to the drone, and then to the ground station is the optimal solution for the characteristics of our mission.

The final stage of the project was to integrate the different components together as well as ensuring the correct operation of the software for the complete system. After this stage, the full electrical system was subjected to tests which showed very positive results, as a functionality over 80% was achieved. Nevertheless, the sensor pack as a whole displayed some issues when tested underwater and therefore a total success of the sensor pack system was not accomplished. Moreover, further development would be required in order to guarantee that the sensor pack manufactured is reliable and is ready to operate under real conditions.

Acknowledgements

Foremost, I would like to thank my colleagues Maria Atienza and Mihai-Cristian Botea for the amazing work we did together in the past five weeks, for their optimism and support in critical moments along the project and the great perseverance shown. I truly consider that the results presented in this report, as well as the results obtained by the electrical system of the sensor pack reflect equal contributions of our work and achievements.

I would also like to thank my Supervisors Dr. Thulasi Mylvaganam and Dr. Aditya Paranjape for their encouragement and great advise and guidance, Dr. Mirko Kovac for the possibility of working in such a challenging and interesting project and the entire team of supervisors involved in this project, who have been always willing to help, guide and share their experiences. Lastly, I would like to thank the other members of the sensor pack team: Elinor Merkier, Hasan Rahhal and Zeyad Sakr, for their involvement in the project, their great efforts and work ethic and the uncountable hours working and testing together.

Contents

1	Introduction	1
2	Electrical System (Hardware)	1
2.1	Board & sensors selection	1
2.2	Battery selection	2
2.3	Minor components	2
2.4	Buoyancy calculations	2
2.5	Workshop work	3
3	Computing System (Software)	3
3.1	Main Mission Algorithm	4
3.2	On-board data processing	5
4	Communications	7
5	Ground Station Processing	9
6	Results	9
7	Discussion	11
8	Conclusion	12
9	Appendix A - Main code	14
10	Appendix B - Matlab Scripts	17
10.1	ROS environment setup	17
10.2	Applied function	18

List of Figures

1	Multiple connections performed in the Arduino board shield as required for the correct operation of the system.	3
2	Central computing and sensing algorithm used for operating the entire sensor pack. (Initially developed by my colleague Mihai-Cristian Botea).	4
3	Diagram of the inner algorithm placed in the code in order to gather and process the data, converting it into a string.	7
4	Diagram of the communication structure between the sensor pack and the ground station.	8
5	Diagram of the algorithm performed by the Matlab function.	9
6	Picture of the assembled electric system before being placed inside the sensor pack case.	11

List of Tables

- 1 Breakdown of the current drawn from the battery by the different components. 2

1 Introduction

Concerns regarding the impact of human activities in nature has evolved substantially in the past decades. The need for a change in the means of obtaining energy, as well as in the way in which all the industries operate and dispose their waste has triggered different organizations and -globally- the society to develop methods of controlling the evolution of affected environments.

One of the main issues arise due to the effects that different industries have in the water. These can be produced by chemical or oil spills, as well as bacteria infections. The substantial costs involved in sensing and sampling water in such conditions and the need for a team of people working together reduces significantly the quality of the inspections and tests conducted, therefore precluding an effective possible reaction to the problem.

The aim of the project is to design, build and test a UAV with autonomous water sampling and sensing capabilities in order to facilitate water monitoring for such conditions, therefore demonstrating the potential of these devices in the industry and as a tool against environmental hazard [1]. Furthermore, this report will be mainly focused in the development of the sensor pack included in the drone and, more specifically, in the electronic and computing system integrated on it. Such system had the main objectives of providing a reliable and systematic method of sensing different parameters in the water and collecting a small sample to be further analyze in the laboratory, as well as allowing live data transmission, processing and representation through the ground station.

2 Electrical System (Hardware)

2.1 Board & sensors selection

The core of the computing and electrical system incorporated in the sensor pack was chosen to be an Arduino Mega 2560, which provided us with enough ports and computing power to process the readings of six different sensors and to control both the motor of the clamping mechanism and the pump. The Arduino Uno was also explored as an option as it is a cheaper and smaller. However, it does not provide us with the number of analog ports required for the mission, and hence it was discarded. Furthermore, budget constraints were also considered in the selection process, thus simplifying the decision as the Arduino Mega is a cheaper solution as compared to other alternatives such as the Raspberry Pi, and still meets all of our requirements. The six sensors are in charge of measuring the temperature, pH, turbidity, electrical conductivity, dissolved oxygen and oxidation reduction potential (ORP) of the fluid. They were specifically chosen because they are the most important parameters for sensing the quality and potability of the water, while allowing us to carry a basic analysis of the other two scenarios studied: an oil spill and aquatic life sustainability [2]. Along with these six sensors, a seventh sensor is used to measure the depth of the pack by using pressure differentials. This is required to control the depth at which the sensing and water sampling is carried, thus adding accuracy to the whole process and enabling the selection of a desired depth beforehand.

2.2 Battery selection

The battery chosen for powering the Arduino board and the two motors is a 11.1V 3-Cell Li-Po battery with a capacity of 450 mAh. It was selected after carrying the calculation of the total current drawn from the battery by the Arduino board, the sensors and the motors. The accessibility of the battery was also a concern and therefore it was decided that the battery should last up to three or four flights before requiring a recharge, hence reducing the frequency at which the sensor pack must be disassembled. The following table summarizes the drawn current calculations performed, which gave an estimate of 28 minutes of battery life at maximum consumption rate. However, maximum consumption rate is only achieved during approximately 30 seconds on each mission and therefore the battery life is substantially longer than the estimate.

Table 1: Breakdown of the current drawn from the battery by the different components.

Component	Quantity	Current (mA)
Analog Pin (Arduino)	3	50
Digital Pin (Arduino)	1	40
Serial Pin (Arduino)	2	50
Pump Motor	1	300
Clamp Motor	1	80
Total Current Drawn		670

2.3 Minor components

The Arduino Mega is a relatively small board and therefore it has its limitations. The most important one is its incapacity to provide the two motors with the required current, hence resulting in the motors having to draw the current directly from the battery by using a voltage regulator and having to be driven by a Brushed Motor Driver chip, the STMicroelectronics L293D, along with two capacitors of $47\mu\text{F}$ and $100\mu\text{F}$, also included in the circuit with the purpose of noise reduction. Moreover, a shield for the Arduino board is also used in order to facilitate the placement of the different connections as well as to provide enough ports to connect all the sensors to the ground and power inputs as required. This option was selected over a protoboard because it is easier place, it remains fixed with no issues and it saves a substantial amount of volume.

2.4 Buoyancy calculations

This last quality is very important as buoyancy calculations carried at the initial stage of the project showed that the total volume of the sensor pack exceeded the allowed one for the mass of the sensor pack to be able to sink, thus risking the entire mission. These calculations were carried using Archimedes' Principle given by:

$$F_A = \rho_w g V \quad (1)$$

Where F_A is the buoyant force, ρ_w is the density of the liquid (water in our case) and V is the volume of water displaced (ie. volume of the sensor pack). Thus the buoyant force must be less than the weight of the sensor pack calculated as $W = mg$ in order for the sensor pack to sink [3].

2.5 Workshop work

Due to the complexity of the hardware belonging to the sensor pack, a big amount of time was spent in the electronic workshop soldering and integrating all the components together in order to achieve the desired system, which in total has more than 50 connections and 10 individual circuits. This was a big challenge for us, as our skills had to be developed fast and deep in order to possess the required level of ability and comprehension for the system to function properly and without faults. The Arduino shield belonging to the electrical system is shown below.

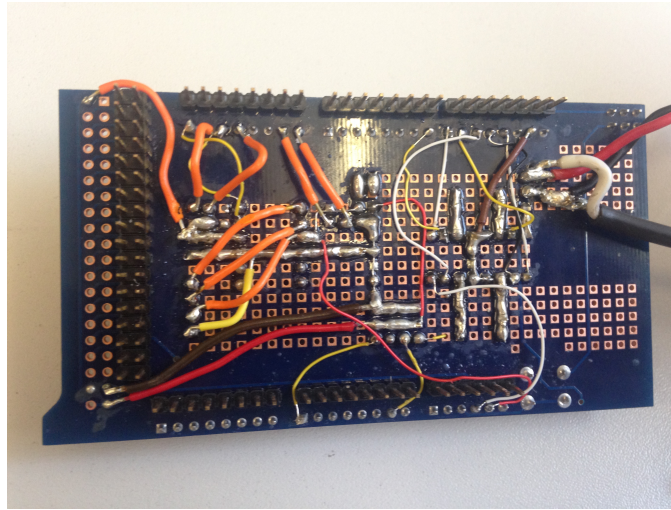


Figure 1: Multiple connections performed in the Arduino board shield as required for the correct operation of the system.

3 Computing System (Software)

The other critical system in the sensor pack is the software, which is closely integrated and related to the hardware structure. The main objectives of such system are to process the information received from all the components in an efficient, systematic manner and being able to take action as required. This includes driving the motors, gathering the required data through the sensors, control the depth of the sensor pack and being able to stop the entire process if a water leak inside the sensor pack is detected [4].

The programming involved was required to be carried in C++, as it is the language used by the Arduino Mega board. The code was written in the Arduino IDE, as it was determined to be the most reliable option when uploading the sketch (ie. script) to the board and displaying the data through the serial monitor. This was crucial in order to debug the

code, as it was easy to check the measurements and the processing conducted by the Arduino board by using the serial monitor.

3.1 Main Mission Algorithm

The program involves an algorithm that ensures the correct functioning of the entire water sensing and sampling system. It was developed at the early stages of the design and elements were added into it as the project moved forward and the sensors were acquired, nevertheless keeping the same structure throughout. This algorithm is better explained in the form of a diagram, which is shown below.

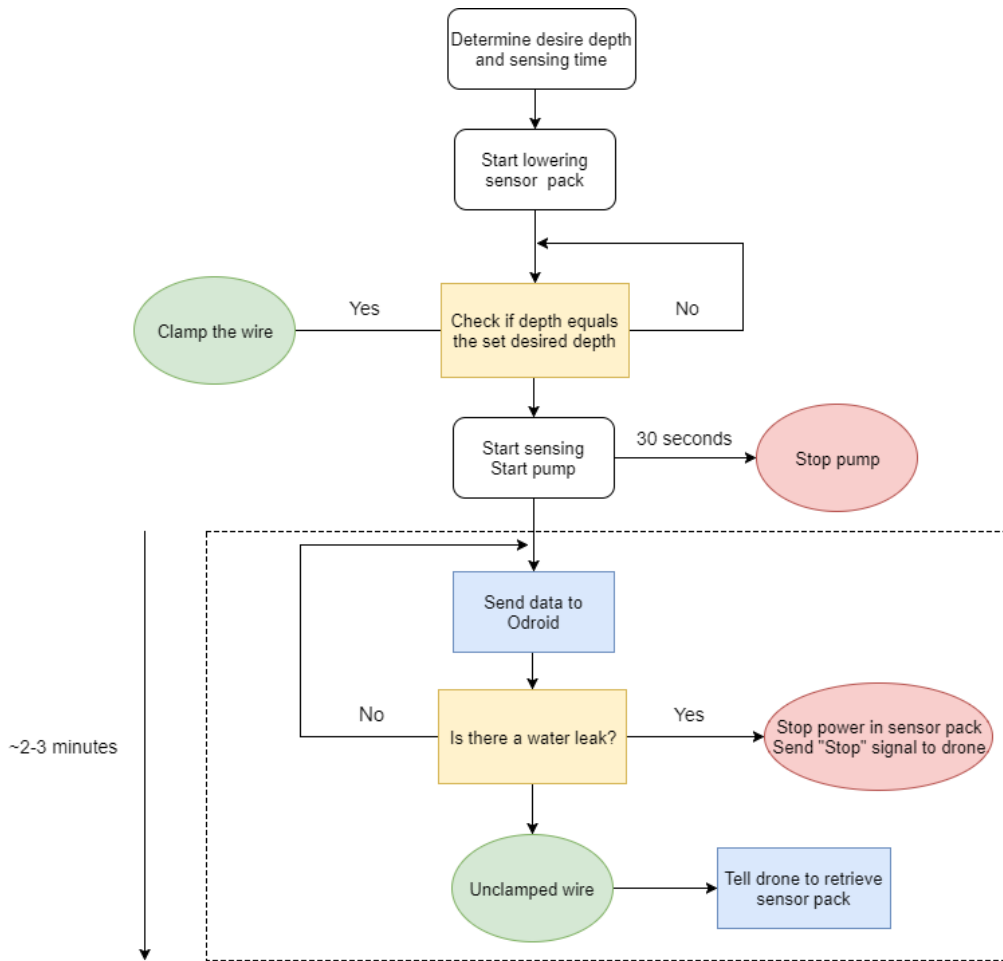


Figure 2: Central computing and sensing algorithm used for operating the entire sensor pack. (Initially developed by my colleague Mihai-Cristian Botea).

As can be seen from the diagram, the sensing process starts by specifying in the code the desired depth and the amount of time that is going to be used for sensing. When the drone reaches the position at which the water monitoring must be conducted, it starts lowering the sensor pack. During this process, the depth is measured and compared with the reference one. When the depth of the sensor pack reaches the desired depth, the Arduino Board tells the clamping mechanism motor to operate during 2 seconds (to clamp the wire). Right after this, the Arduino board starts reading the measurements from the different sensors and tells the pump to start driving water into the container. The process

of sampling water lasts 30 seconds, which is the time needed for the pump to push the right quantity of water (50 ml. for the test) into the container bag. However, the sensing process extends to two minutes (for the test, in operating conditions it should be more). The reason is that the sensors need approximately two minutes in order to sense a stable and converging value, and hence data gathered before that time cannot be entirely trusted and must be considered carefully while conducting our analysis. When the sensing time specified in the code is reached, a signal is sent to the motor in the clamping mechanism and the wire is unclamped. The sensor pack is then retrieved and the drone can return to the ground station.

Along with the water quality monitoring, a water leak sensor is always active and operating, from the moment that the drone is in the air, to ensure that possible failures in the sensor pack case do not result in a catastrophic damage to the electrical hardware inside. If the water leak sensor detects water, then the Arduino sends a message to the drone and the sensor pack is immediately retrieved and the power on the electrical system is stopped. The correct operation of this part of the algorithm is critical, as a malfunction could result in a partial or total loss of the system and thus a loss of a considerable amount of money.

3.2 On-board data processing

The Arduino Mega is in charge of receiving the readings from the sensors and processing them in order to have a valid set of data before it is sent to the ground station. The Baud rate chosen for the serial, taking into consideration the sensors used as well as ensuring that no important data is lost, is of 9600 (bits per second). The pH, turbidity and electrical conductivity sensors send the information to the board through analog pins, while the temperature and depth sensors use digital pins in order to transmit the information. For the case of the dissolved oxygen and oxygen reduction potential sensors, the data is received through serial ports in the Arduino board. After the different readings are received, they must be converted into the desired parameter. The pH analog reading is converted to a voltage, which is then converted into the pH value. In the turbidity case, the analog reading is also converted to the voltage required for turbidity analysis. The conversion of analog readings to a voltage value is given by:

$$Voltage = \frac{r_A \times 5}{1024} \quad (2)$$

Where r_A is the analog reading value and *Voltage* is the desired value in volts [6]. For the two sensors involving serial ports (ie. dissolved oxygen and oxygen reduction potential), the readings do not need to be converted and therefore no additional processing is required.

However, the electrical conductivity sensor is more complicated and requires of a careful analysis and comprehension. In order to achieve the desired parameter, the Arduino Board must use the temperature reading from the temperature sensor and input it as a variable in the electrical conductivity processing stage. The reason behind this is that the conductivity varies with temperature, and a compensation factor is required for obtaining the correct value. This factor is calculated as:

$$K_T = 1 + 0.0185 \times (T - 25) \quad (3)$$

Where K_T is the temperature compensation factor and T is the current temperature as read from the temperature sensor. Then, the voltage coefficient K_V can be calculated as:

$$K_V = \frac{V_a}{K_T} \quad (4)$$

Where V_a is the average voltage resulting from the conversion of the analog readings using equation 2 and K_T is the temperature compensation factor mentioned before.

Along with this calculations, a median filter is used in order to ensure that a stable value is achieved before the information is transmitted by the Arduino. This non-linear filter has the ability to reduce the noise in the signal received from the sensor and therefore improves the results of later processing. Moreover, the formula to obtain the final value of the electrical conductivity varies according to the voltage coefficient, since for different ranges of this value, different constants must be used [5].

$$\kappa = 6.84 \times K_V - 64.32, \quad \text{for } K_V \leq 448 \quad (5)$$

$$\kappa = 6.98 \times K_V - 127, \quad \text{for } 448 < K_V \leq 1457 \quad (6)$$

$$\kappa = 5.3 \times K_V + 2278, \quad \text{for } 1457 < K_V \leq 3300 \quad (7)$$

These factors are found through the calibration process of the sensor, which results in different sets of constants depending on the voltage coefficient of the solution (ie. liquid) that is going to be analyze.

After all the readings from the different sensors are processed for a single timestep, they are converted into a format feasible for communications. The optimal format for our requirements was found to be a single string, with the different values delimited by a space and, in the last place, the timestep at which that set of values had been measured. This last value was included in order to simplify the processing at the ground station, which will be explained later on the report. Therefore a single string of data is built by merging the values from the different sensors at each timestep. When the data is transmitted, the sensing loop starts again and a different string is build and sent from the Arduino. This process is repeated until the maximum time specified beforehand in the code is reached. The output string has the form:

$$S_{op} = [s_{depth} \ s_1 \ s_2 \ s_3 \ s_4 \ s_5 \ s_6 \ \Delta t] \quad (8)$$

Where s_n are the readings from the six different sensors and Δt is the timestep. It is also important to mention that the depth is transmitted in order to be able to clamp the motor, and a message from the water leak sensor intercalates with the strings in the serial monitor. This is done in order to be able to stop the mission if water inside the sensor pack is detected. However, this last message is not sent to the ground to be processed. A diagram of the code algorithm is shown below.

For the part of the code related to the actuator motor in charge of clamping the wire, a

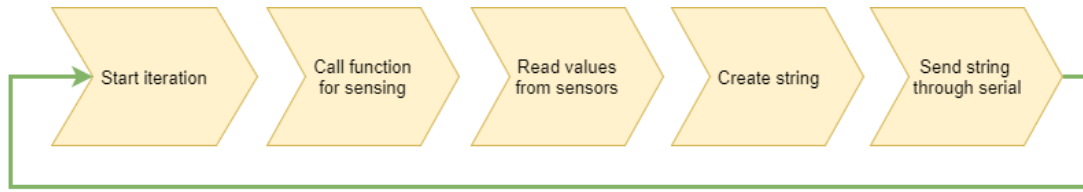


Figure 3: Diagram of the inner algorithm placed in the code in order to gather and process the data, converting it into a string.

running time of two seconds is specified. This time was set by the Hardware team and it is the required time for the actuator to clamp the wire and, after the sensing stage is completed, unclamp the wire again and return to the initial position. In the case of the pump, a test was carried in order to determine the amount of time required to fill the container bag and hence collect the sample of water. This time was found to be 30 seconds for 50 ml. as mentioned before. After this time is reached, the motor is stopped by the Arduino board. Note that the code may be found in the appendix of this report.

4 Communications

A very important requirement specified for the mission was the live data transmission, processing and display. Therefore the communications between the sensor pack, the drone and the ground station play a very important role in the success of the mission. As a result, the options and methods for transmitting the information from and to the sensor pack were carefully studied and analyzed.

The first step in the development was to determine how the link between the sensor pack (transmitter) and the ground station (receiver) was going to be. From initial considerations it was determined that the information could not be sent directly from the transmitter to the receiver. The reason is that, since any type of transmitter should be placed on the sensor pack, it would be operating underwater. However, the only feasible type of wireless communication for our mission profile and budget would be Wi-Fi. This is a critical issue due to the high conductivity of water, which results in a high absorption of radio waves, therefore making impossible to communicate through Wi-Fi when the sensor pack is underwater [7].

The incapacity of using Wi-Fi underwater led to the second option considered, which was based on transmitting the data first to the drone and then send it from the drone to the ground station. As before, wireless communications between the sensor pack and the drone were considered. However, although options such as very low radio frequencies as the ones used in submarines were studied, it was concluded that the only feasible option for wireless communications was bluetooth. Nevertheless, this type of communications has the same problems as Wi-Fi, and therefore it is not able to work underwater. As a result, wireless data transmission from the sensor pack was discarded as an option, hence leaving just one option for live data transmission: the use of serial communications (ie. by wire) from the Arduino serial ports to the drone and then through Wi-Fi from the drone to the ground station.

For this method to work, a transmitter capable of receiving the data from the Arduino

board -in the sensor pack- and sending it through Wi-Fi to the ground station must be placed in the drone. However, in order for the drone to achieve autonomous flight, the Flight Controls team integrated an Odroid board in the drone, which is also capable of handling the communications between the sensor pack and the ground station. In that way, there was no need for another system in the drone, thus saving space and weight and cutting costs. Hence the communication diagram resulted as shown below.

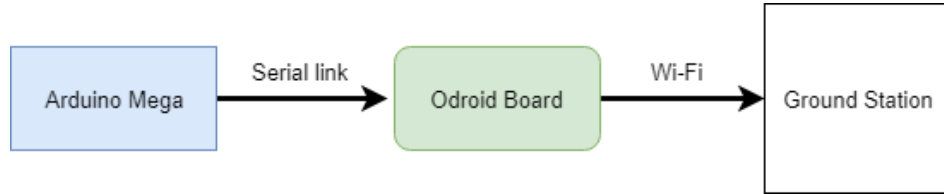


Figure 4: Diagram of the communication structure between the sensor pack and the ground station.

Once the channel of communication was selected, the next step was to determine the format in which the data should be transmitted. As the development of the project advanced, it became clear that this step was critical for live data processing and plotting. The initial option was to use a handshake-based protocol in which, after one string of data is sent from the Arduino to the ground station, a byte is sent in the opposite direction in order to inform the Arduino processor that the data has been received, and that it is ready to receive another set. This operation ensures that no mistakes occur in the communications, as the returning byte acts as feedback and it is only sent if the data is received correctly. However, the Flight Controls team was in charge of handling the communications between the Odroid and the ground station and, after carefully studying this option, both groups decided to discard it, as sending a byte of data in the opposite direction (ie. from the ground station to the sensor pack through the Odroid) was not an easy task.

Therefore the only option left was to send the string of data built in Arduino one at a time. However, this option requires a more careful consideration of the form in which the data is sent, since a mistake in the order in which the data from each sensor is placed in the string and received by the ground station would result in a critical failure of the live data processing capabilities of the system. Another possible issue would be a mishandle of the information by the Odroid board, thus resulting in the same failure.

The communication between the Odroid board and the ground station is done through a Robotic Operative System (ROS) environment, which is also used for the flight controls communications and provides a reliable and simple integration with the systems in the ground station. This environment is based on topics that operate through Wi-Fi. The topic is setup beforehand and when the transmitter system is ready to send data, it publishes this data in the topic created. Then, the receiver system can subscribe to the topic - providing that they are connected to the same network and the IP addresses are known- and receive the message in the same exact way as it was published by the transmitter system. With this method, it is possible to ensure that the information is not modified in the communication process, hence reducing the probability of a failure. Furthermore, the ROS environment is capable of working with strings of data, reason why the information from the sensors is converted to a string in the Arduino board before being sent. Therefore building the correct string of information containing all the measurements from the sensors

is the only requirement for the channel of communication between the Arduino and the ground station to operate properly [8].

5 Ground Station Processing

Another advantage of using the ROS environment is that it can be easily implemented in processing software such as Processing, Matlab or Simulink. Although other options were also considered for data processing, it was decided to use Matlab or Simulink, since we have three years of experience and we are familiarized with the environment, hence reducing the amount of time required to setup the software and simplifying the process, while retaining all the capabilities required. Furthermore, after various tests it was decided to use Matlab for processing the data rather than Simulink, since it was shown to be simpler to configure and manage, as well as it allows a higher degree of modifications.

In Matlab, it is required to set and initiate the ROS environment by connecting the transmitter and receiver to the appropriate network and inputting the IP addresses of the computer receiving the information and the Wi-Fi module connected to the Odroid board. After this process is finished, Matlab is able to continuously receive the string of data as sent by the Arduino with almost no delay.

Therefore, the only step left is to process the string of data received through the ROS topic. In order to do this, it is possible to create a function in Matlab and apply it directly to the incoming messages. This is the main advantage of using Matlab for processing ROS messages, as no other commands or scripts are required apart from the function built. In the first tests, a simple function was used in order to check that the information was received properly. However, the complexity of the function was increased in later stages of the development. The main objective of the function is to store the data received from the sensors in an orderly manner, storing the information from each individual sensor separately in the form of an array, and to plot the data as it is received from the sensor pack in individual graphs [9].

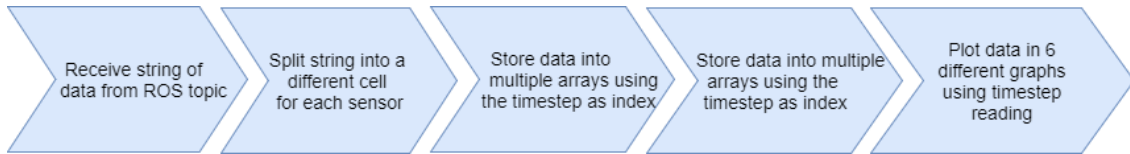


Figure 5: Diagram of the algorithm performed by the Matlab function.

6 Results

Regarding the performance of the entire sensor pack, there were a few important issues during the stages previous to testing. The sensor pack case was designed in order to enclose as little volume as possible while guaranteeing that the electronic system could fit inside and that no water could go inside and damage the components. However, the optimization was pushed beyond the limit and as a result, when we tried to fit the electric system with all the probes, the board, the wires and the battery inside the case we realized that the total volume occupied by the system was underestimated in the calculations. Hence it was

not possible to place the entire system inside the sensor pack case in a comfortable manner. However, after hours of rearranging the different components of the system, we managed to fit the entire electric system inside the case.

The second issue, discovered during preliminary testing, was a failure of the waterproofing capabilities of the sensor pack. The case was 3D printed using Nylon as the material, which in theory was the best option for the design developed and for saving weight. However, during testing it was discovered that the nylon structure could absorb water and as a result, small water leaks inside the sensor appeared. Moreover, the glue used to place the bottom part of the sensor pack case and its lid together could not isolate the interior of the sensor pack properly, thus resulting in the possibility of water coming inside.

Regarding the electric and computing system, it can be stated that the integration and later test were a success, since the system was working correctly and as expected. The only fault detected during the test was the no possibility of integrating the electrical conductivity sensor software in the main script written in Arduino. The reason behind this fault lies in the complexity of the sensor, as explained in the previous section. It was possible to use the sensor and make it work on its own, but some critical issues arose when trying to implement it as a function in the main code. Nevertheless, a full test including the actuator motor, the pump and the rest of the sensors was conducted. The test reflected a full agreement between the expectations while building the script and the results, hence achieving a total success in terms of software. The motor and the pump exhibited a correct operation as specified in the code and the data from the sensors was found to be in accordance to previous experiments and known values for the different parameters. Furthermore, it is possible to state that the main algorithm for the entire sensing and water system explained in previous sections worked without any issue, since the script was able to process the information received through the depth (pressure) sensor and act accordingly.

The data transmission between the sensor pack and the ground station was partially tested, reflecting very positive results when connecting the serial link to a computer, therefore modifying the intermediate step of passing the information through the Odroid and pass it through a computer instead. We are expecting to test the full system including the Odroid in the next few days. However, the tests conducted are a very good simulation of the real transmission, since the Wi-Fi connection and ROS environment were also successfully tested. We were able to process and display the data received through the ROS node in Matlab correctly, even if plots were not produced in the preliminary tests. Nevertheless, the more complex part is the data transmission and hence, since the later was operating with no issues, the plotting script should work as designed. It is also important to mention that a filter was added as an intermediate step, since the signal displayed some noise when received by the computer. This filter was implemented by the Flight Controls team, as they were in charge of the Odroid and its communications, and were more familiarized with its environment.

The resulting hardware of the electric system was also successfully tested, as no faults or issues appeared in the connections made by wire or in the different components such as the actuator motor, the pump or the sensors, which were operating correctly and as expected. This result reflects the big amount of hours spent in the electronic workshop assembling the system carefully and methodically. A small issue appeared in preliminary

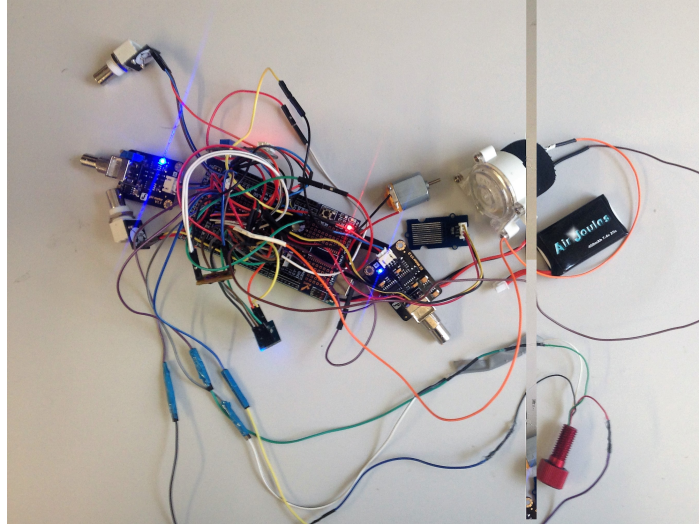


Figure 6: Picture of the assembled electric system before being placed inside the sensor pack case.

tests of the full system, as a part of the Arduino Mega board got slightly overheated when the motors were in operation. However, under the supervision of Dr. Franco Giammaria it was concluded that the issue could not produce an appreciable impact in the system and therefore that it would not involve any risk. As a result, it was decided not to make any further modifications to the system. A picture of the complete system is shown above.

7 Discussion

Although the tests of the electric and computing systems were successful, major improvements could be made in order to have a more reliable system. The electrical conductivity sensor could not be integrated in the algorithm in time for the testing stage. However, with more time to understand and develop its functionality, we think it would be possible to achieve the complete operation of the sensor, and therefore of the entire algorithm. Furthermore, a more sophisticated board could be used instead of the Arduino Mega in order to increase the protection of the system, since this type of board can be easily damaged if one or more connections are not done properly. However, a more expensive option was not feasible giving the budget constraints of the project.

With reference to the communications and data processing in the ground station, a full test of the data transmission capabilities of the integrated system would be very helpful in considering possible improvements. Nevertheless, it is possible to state that the implementation of the hand-shake protocol would result in a more reliable transmission and therefore it would have a positive impact. This could be possible providing an extended time frame for the project. Furthermore, the Matlab model and function could be further optimized for better data visualization and efficiency, such as allowing the measurements to be processed and represented with no need of adding the timestep value to the string in the Arduino script.

Regarding the entire sensor pack system, it was clear from the test results that major upgrades would be required in order to achieve a successful and complete mission as initially

planned. First of all, the enclosed volume of the sensor pack case should be increased in order to leave more room for placing the electric system, which otherwise would not fit properly. Furthermore, another material should be selected for 3D printing the case, since the nylon showed water absorption properties that could lead to a water leak and thus result in irreversible damage to the electric system. Improving the waterproof method for the different openings in the case such as the probes holes and the lid is also a pending task, since tests showed that water could enter through the space between the lid of the sensor pack and the sensor pack itself, and the solution adopted for the probes holes using silicon is rather crude. A possible solution would be to use a different type of glue specifically design for underwater conditions in order to ensure a correct isolation when the lid is fixed to the sensor pack. Moreover, a cleaner method could be used to waterproof the probes holes, such as removable rubber rings. This would also simplify the disassembling and maintenance of the sensor pack, as probes could be more easily removed and replaced.

8 Conclusion

The final tests showed that the electrical system built could perform without issues under normal circumstances, although full operation was not achieved due to the problem with the electrical conductivity sensor. Furthermore, the system proofed to be reliable and robust, since it managed to perform in a consistent basis and to obtain sensible data valid for the analysis and monitoring of the water.

However, the integration of the sensor pack was not fully successful, as it displayed a lack of space for placing the entire electrical system in a safe and reliable manner. Different options were studied in later stages of the development, when the fault was encountered. Nevertheless, the amount of time required for the 3D printer to manufacture new prototypes dragged the development in the last week, precluding the testing of new designs.

Overall, more emphasis should be placed in selecting the appropriate materials for manufacturing the prototypes, as many aspects of the design such as waterproofing capabilities and weight could be improved and optimized. However, the range of materials for 3D printing is limited, and therefore this should also be considered.

The use of unmanned aerial vehicles could soon have a major impact in the way certain tasks are handled. The potential for this devices in highly structured and systematic procedures such as water monitoring and sampling is enormous, offering a cheaper and more reliable option as well as reducing the human resources needed. Nevertheless, further research and development should be carried in order to achieve highly efficient and effective devices for use in a daily basis.

Even though a fully operational system was not successfully achieved, I consider that the work and results presented in this report, as well as the results obtained for the overall project show a substantial contribution to the development of unmanned aerial vehicles for field-based tasks such as water sensing and sampling. Hopefully the work presented can be used as initial reference for future projects and help in the development of this type of technologies.

References

- [1] Cengiz Koparan. *Unmanned Aerial Vehicle (UAV)-Assisted Water Sampling*. Clemson University. (Accessed on 19th May 2017).
- [2] Department of Water. *Surface ater sampling methods and analysis- technical appendices*. Government of western Australia (Accessed on 19th May 2017).
- [3] Benny Lautrup. *Physics of Continuous Matter*. The Niels Bohr Institute (Accessed on 28th May 2017).
- [4] John Boxall. *Arduino Workshop*. No Starch Press (Accessed on 2nd June 2017).
- [5] DFRobots Electronics. *Analog electrical conductivity sensor documentation*. (Accessed through: [http : //www.dfrobot.com/wiki/index.php/Analog_{EC} meters_{KU} : DFR0300](http://www.dfrobot.com/wiki/index.php/Analog_EC_meters_KU_DFR0300) on 10th June 2017).
- [6] SparkFun. *Analog to digital conversion*. (Accessed through: <http://learn.sparkfun.com/tutorials/analog-to-digital-conversion> on 10th June 2017).
- [7] Milika Stojanovic. *Underwater wireless communications: current achievements and research challenges*. Massachusetts Institute of Technology. (Accessed on 4th June 2017).
- [8] ROS.org. *Communications*. Robotics Operative System Documentation. (Accessed through: <http://wiki.ros.org/ROS/Patterns/Communication> on 9th June 2017).
- [9] Mathworks. *Matlab ROS environment initialization*. MATLAB ROS documentation. (Accessed on 13th June 2017).

9 Appendix A - Main code

```
1  //*****Depth initialization*****
2  #include <Wire.h>
3  #include "MS5837.h"
4  MS5837 sensor;
5  #define WATER_SENSOR 2
6  //*****
7
8  //*****FLAGS*****
9  #include <elapsedMillis.h>
10 elapsedMillis timeElapsed;
11 int desired_depth = 40;
12 float depth_check;
13 boolean depth_reached = false;
14 boolean clamp_flag = false;
15 boolean unclamp_flag = false;
16 boolean done_flag = false;
17 //*****
18
19 //*****Motor initialization
20 int enableClamp = 12; // PWM
21 int MotorA1 = 11; // PWM
22 int MotorA2 = 10; // PWM
23
24 //Pump motor (B)
25 int enablePump = 8;
26 int MotorB1 = 9;
27 int MotorB2 = 10;
28
29
30 //*****pH initialization*****
31 #define SensorPinpH 0 //pH meter Analog output to Arduino Analog Input 0
32 unsigned long int avgValue; //Store the average value of the sensor feedback
33 float b;
34 int buf[10],temp;
35 //*****
36
37 //*****ORP initialization*****
38 String sensorstring = "";
39 boolean sensor_string_complete = false;
40 //*****
41
42 //*****DO initialization*****
43 String DOstring = "";
44 //*****
45
46
47 void setup()
48 {
49     Serial.begin(9600);
50     Serial3.begin(9600);
51     Serial1.begin(9600);
52     sensorstring.reserve(30);
53     DOstring.reserve(30);
54     Wire.begin();
55     initDepth();
56     pinMode(enableClamp, OUTPUT);
57     pinMode(MotorA1, OUTPUT);
58     pinMode(MotorA2, OUTPUT);
59     pinMode(enablePump, OUTPUT);
60     pinMode(MotorB1, OUTPUT);
61     pinMode(MotorB2, OUTPUT);
62 }
63
64 void serialEvent3(){
65     sensorstring = Serial3.readStringUntil(13);
66     sensor_string_complete = true;
67 }
68
69 void serialEvent1(){
70     DOstring = Serial1.readStringUntil(13);
71 }
72
73 void loop()
```

```

74 { float depth_check = getDepth(); Serial.print(",");
75 if (depth_check >= desired_depth){
76     depth_reached = true;
77 }
78 clamp();
79 if (depth_reached == true and clamp_flag == true and done_flag == false){
80     timeElapsed = 0;
81     while (timeElapsed <= 120000){
82         //String stDepth, stTemp, stTurb, stpH;
83         sense();
84         leaks();
85     }
86     done_flag = true;
87     Serial.println("START SAMPLING.");
88     pump();
89     Serial.println("DONE.");
90     unclamp();
91 }
92 leaks();
93 delay(500); }
94
95 void initDepth()
96 {
97     // Initialize pressure sensor
98     // Returns true if initialization was successful
99     // We can't continue with the rest of the program unless we can initialize the
    sensor
100 while (!sensor.init()) {
101     Serial.println("Init failed!");
102     Serial.println("Are SDA/SCL connected correctly?");
103     Serial.println("Blue Robotics Bar30: White=SDA, Green=SCL");
104     Serial.println("\n\n\n");
105     delay(5000);
106 }
107 sensor.setModel(MS5837::MS5837_30BA);
108 sensor.setFluidDensity(997); // kg/m^3 (freshwater, 1029 for seawater)
109 }
110
111 float getDepth(){
112     //*****Read data from depth sensor and convert to strings*****
113     sensor.read();
114     String strDepth;
115     float floatDepth=sensor.depth();
116     if (floatDepth>=0){
117         double prec_Depth=100*floatDepth;
118         int intDepth=(int)prec_Depth;
119         String strDepth=String(intDepth);
120         if (intDepth<100){
121             strDepth="0"+strDepth;
122             if (intDepth<10){
123                 strDepth="0"+strDepth;
124             }
125         }
126         Serial.print(strDepth);
127     }
128     else {Serial.print("000");}
129     return 100*floatDepth;
130     //return floatDepth;
131 }
132
133 String getTemp(){
134     //*****Read data from depth sensor and convert to strings*****
135     sensor.read();
136     String strTemp;
137     float floatTemp=sensor.temperature();
138     if (floatTemp>=0){
139         double prec_Temp=10*floatTemp;
140         int intTemp=(int)prec_Temp;
141         String strTemp=String(intTemp);
142         if (intTemp<100){
143             strTemp="0"+strTemp;
144             if (intTemp<10){
145                 strTemp="0"+strTemp;

```

```

146     }
147   }
148 }
149 else {strTemp = "000";}
150 return strTemp;
151 }
152
153 String getTurb(){
154   int TurbValue = analogRead(A1); // read the Turbidity input on analog pin 1
155   float voltage = TurbValue * (5.0 / 1024.0); // Convert the analog reading (which
    goes from 0 - 1023) to a voltage (0 - 5V);
156   double prec_voltage=100*voltage;
157   int intvoltage = (int)prec_voltage;
158   String strvlt = String(intvoltage);
159   if (intvoltage<100){
160     strvlt="0"+strvlt;
161     if (intvoltage<10){
162       strvlt="0"+strvlt;
163     }
164   }
165   return strvlt;
166 }
167
168 String getpH(){
169   for(int i=0;i<10;i++) //Get 10 sample value from the sensor for smooth
    the value
170   {
171     buf[i]=analogRead(SensorPinpH);
172     delay(10);
173   }
174   for(int i=0;i<9;i++) //sort the analog from small to large
175   {
176     for(int j=i+1;j<10;j++)
177     {
178       if(buf[i]>buf[j])
179       {
180         temp=buf[i];
181         buf[i]=buf[j];
182         buf[j]=temp;
183       }
184     }
185   }
186   avgValue=0;
187   for(int i=2;i<8;i++) //take the average value of 6 center
    sample
188     avgValue+=buf[i];
189   float pHValue=(float)avgValue*5.0/1024/6; //convert the analog into millivolt
190   pHValue=3.5*pHValue; //convert the millivolt into pH value
191   double prec_pHValue=100*pHValue;
192   int intpHValue = (int)prec_pHValue;
193   String strph = String(intpHValue);
194   return strph;
195 }
196
197 void leaks(){
198   if(digitalRead(WATER_SENSOR) == LOW){
199     Serial.print("STOP!!! ");
200   }
201   else{
202     Serial.print("NO LEAK ");
203   }
204 }
205
206 void clamp(){
207   if (depth_reached == true and clamp_flag == false){
208     analogWrite (enableClamp, 250);
209     digitalWrite (MotorA1, LOW);
210     digitalWrite (MotorA2, HIGH);
211     delay(2000);
212     analogWrite (enableClamp, 0);
213     clamp_flag = true;
214   }
215 }
216
217 void unclamp(){

```

```

216     analogWrite (enableClamp, 250);
217     digitalWrite (MotorA1, HIGH);
218     digitalWrite (MotorA2, LOW);
219     delay(2000);
220     analogWrite (enableClamp, 0);
221 }
222
223 void pump(){
224     analogWrite (enablePump, 250);
225     digitalWrite (MotorB1, HIGH);
226     digitalWrite (MotorB2, LOW);
227     delay(30000);
228     analogWrite (enablePump, 0);
229 }
230
231 void sense(){
232     float stDepth = getDepth();
233     String stTemp = getTemp();
234     String stTurb = getTurb();
235     String stpH = getpH();
236
237     Serial.print(stTemp+" "+stTurb+" "+stpH+" "+sensorstring+" "+DOstring+"
"+timeElapsed);
238     Serial.println(); //print next iteration in a new line
239     /*Serial.print(sensorstring);Serial.print(",");
240     Serial.print(DOstring);*/
241     delay(500);
242 }
243

```

10 Appendix B - Matlab Scripts

10.1 ROS environment setup

```

%-initialize ROS Network-
setenv('ROS_IP','192.168.137.235');
setenv('ROS_MASTER_URI','http://192.168.217.129:11311');
roscinit;

%subscribe to topic
sensorsub = rossubscriber('/arduinoData',@ReadSensor);

global SensorStr;
global ConvData;

```

10.2 Applied function

```
function ReadSensor(src, msg)
%function applied to the messages received through the ROS node, which
%contains all the data from the sensors, and process it. The processing
%stage consists on storing and plotting the data

%the string received through ROS must be declared as global
global SensorStr;
global ConvData;

SensorStr = msg.Data;           %receive the message
ConvData = str2num(SensorStr); %split the string into an array

%store the data in 6 different arrays
Depth_store(ConvData(7)) = ConvData(1);
Temp_store(ConvData(7)) = ConvData(2);
Turb_store(ConvData(7)) = ConvData(3);
pH_store(ConvData(7)) = ConvData(4);
ORP_store(ConvData(7)) = ConvData(5);
DO_store(ConvData(7)) = ConvData(6);

%    figure(1);
%    hold on
%    plot(ConvData(4),ConvData(7)/1000);
%    h = animatedline(ConvData(7),ConvData(4));
%    xlabel('Time [timestep]'); ylabel('pH')
%    hold off
%
%    figure(2);
%    hold on
%    plot(ConvData(3),ConvData(7)/1000);
%    xlabel('Time [timestep]'); ylabel('Turbidity [V]')
%    hold off
%
%    figure(3);
%    hold on
%    plot(ConvData(2),ConvData(7)/1000);
%    xlabel('Time [timestep]') ylabel('Temperature [C^0]')
%    hold off
%
%    figure(4);
%    hold on
%    plot(ConvData(5),ConvData(7)/1000);
%    xlabel('Time [timestep]'); ylabel('Oxygen Reduction Potential [mV]')
%    hold off
%
%    figure(5);
%    hold on
%    plot(ConvData(6),ConvData(7)/1000);
%    xlabel('Time [timestep]'); ylabel('Dissolved Oxygen [l]')
%    hold off

    disp(SensorStr);
end
```